

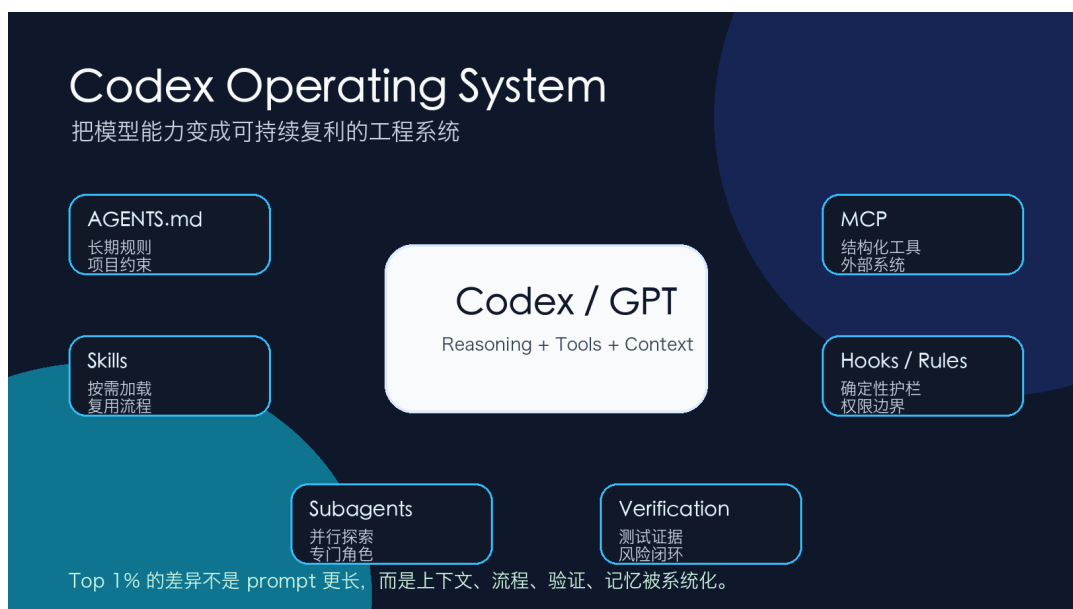
Everything CodeX

副标题: 从熟练使用到 Top 1% Codex Operator 的完整学习材料

版本: v2 draft, 2026-05-17, Asia/Singapore

适用对象: 想把 Codex 当成长期工程能力来训练的开发者、架构师、技术负责人、AI 工程效率负责人、准备对外讲授 Codex 工作法的人。

说明: OpenAI 官方名称是 Codex。本文沿用你给出的 "CodeX" 作为标题风格, 但正文默认使用 "Codex"。



阅读地图

这版把第一版内容重新排成手册结构: 先建立心智模型，再搭建工作台，然后进入 workflow、自动化、安全、评估和授课。

章节组	你要学到什么	推荐产出
0-4	Codex 的心智模型、能力地图、模型选择、与 everything-claude-code 的迁移关系	你自己的 Codex 能力地图
5-10	工作台、AGENTS.md、config、rules、hooks、skills	一个可复用项目模板
11-17	MCP、subagents、worktrees、prompt、CLI、App/IDE/Cloud、GitHub 工作流	一套端到端研发流程
18-22	验证、memory、安全、经典模板、自动化	一套质量与安全闭环
23-27	90 天训练、评分表、授课大纲、个人 playbook、原则	可讲授的课程框架

阅读建议: 不要按功能记忆。每读完一章，都问自己: 这部分应该沉淀成 AGENTS.md、skill、hook、rule、MCP，还是自动化脚本？

0. 这份材料怎么学

这不是一份命令手册。它的目标是让你形成一套可复用、可验证、可教学的 Codex 工作系统。

你要掌握的不是 "问 Codex 写代码"，而是:

- 如何选择最合适的 Codex surface: App、CLI、IDE、Cloud、GitHub、Remote mobile、SDK。
- 如何把一个模糊任务拆成 Codex 能稳定完成的工程任务。
- 如何用 AGENTS.md、config.toml、rules、hooks、skills、MCP、subagents 和 memories 管理长期能力。
- 如何让 Codex 自动化你的高频工作，同时保持安全边界。
- 如何评估自己的 Codex 使用水平，并用 90 天训练到能给别人讲课。

建议学习节奏:

1. 第 1 天读完第 1-5 章，搭建自己的 Codex 工作台。
2. 第 2-7 天只练第 6-12 章的基础工作流，每天至少完成 2 个真实任务。
3. 第 2-4 周开始做 skills、hooks、MCP、subagents 和 worktrees。
4. 第 2 个月开始做自己的评估集、课程讲义和演示项目。
5. 第 3 个月开始把你的经验沉淀成团队规范和公开课。

1. Codex 的正确心智模型

Codex 不是单个聊天窗口，也不是单纯的代码补全。它是一套 agentic engineering harness:

- Codex = GPT 模型能力
- + 本地或云端执行环境
 - + 代码库上下文
 - + 工具调用能力
 - + 权限与沙箱
 - + 持久指令
 - + 技能与子代理
 - + 验证闭环
 - + 记忆与自动化

Top 1% 用户的差异不在于 prompt 更花哨，而在于他们会搭建一套持续复利的系统:

- 任务开始前让 Codex 读对上下文。
- 执行时让 Codex 走正确的工作流。
- 修改后让 Codex 自己验证并给出证据。
- 结束后把可复用经验沉淀为 AGENTS.md、skill、hook、rule 或 memory。
- 下次遇到类似问题时，不再重复解释。

everything-claude-code 给出的核心思路可以浓缩成一句话: 把 agent harness 当作可以优化的工程系统，而不是把它当作一次性聊天工具。

迁移到 Codex 后，这句话要升级为:

把 Codex 当作覆盖终端、IDE、桌面 App、云端任务、GitHub、移动端远程控制和 API 自动化的一套工程操作系统。

2. 2026 年 Codex 能力地图

截至 2026-05-17，Codex 的能力已经不是单点工具，而是多 surface 协作。

Codex Surface Map		
不同入口服务不同任务，不要把所有任务都塞进一个聊天窗口。		
Surface	最佳场景	成熟用法
App	多项目、多线程、review pane	用作日常控制台和任务总览
CLI	本地工程、日志、JSONL、自动化	接入 shell、CI、脚本和 schema 输出
IDE	打开文件上下文、局部修改	边看边改，适合小范围高频协作
Cloud	后台 issue、PR、CI 修复	把明确任务交给远程工程师
GitHub	PR review、@codex fix	把 Codex 嵌入团队研发流程
Mobile	跟进长任务、批准动作	离开电脑也能接管后台任务

原则: 能用 CLI 确定性解决的，不一定上 MCP；需要长期复用的，沉淀为 skill。

Surface	最适合做什么	你要掌握的重点
Codex App	多项目、多线程、本地 worktree、review pane、automations、computer use、remote host	把它当作 Codex 控制台
Codex CLI	深度本地工程、快速 shell 驱动、非交互自动化、JSONL 输出、配置实验	把它当作工程代理终端
Codex IDE Extension	在 VS Code、Cursor、Windsurf、JetBrains 里边看边改	把它当作代码审阅和局部修改搭档
Codex Cloud/Web	后台长任务、云端环境、GitHub repo、PR 生成	把它当作可并行的远程工程师
GitHub Integration	PR review、@codex review、@codex fix、CI 修复	把它接进团队研发流程
Mobile Remote	手机端查看进度、批准动作、补充指令、看 diff 和测试结果	把长任务变成可随时接管的后台工作
Codex SDK / exec	程序化调用、CI、脚本化分析、结构化输出	把 Codex 接入自动化管线

官方文档对 Codex 的描述是: 它可以读、改、运行代码, 并帮助写代码、理解代码、review、debug 和自动化研发任务。官方 Quickstart 也明确 Codex 覆盖 App、IDE、CLI 和 Cloud。

3. 模型选择: GPT-5.5、GPT-5.4、GPT-5.3-Codex 怎么用

官方 Codex Models 页面当前推荐:

- **gpt-5.5**: 当前最新前沿模型, 适合复杂 coding、computer use、知识工作和研究 workflow。
- **gpt-5.4**: 更均衡的旗舰模型, 编码、推理、工具使用和 agentic workflow 都强。
- **gpt-5.4-mini**: 更快、更低成本, 适合轻量 coding、探索类 subagent、格式化和小改动。
- **gpt-5.3-codex**: 专门优化过的 agentic coding 模型, 复杂软件工程能力强。
- **gpt-5.3-codex-spark**: 近实时 coding iteration 研究预览, 适合快速交互。

实践上建议这样选:

任务	首选	备选	说明
复杂跨文件实现	gpt-5.5	gpt-5.4 / gpt-5.3-codex	要求完整计划、验证、回滚点
架构设计和方案评审	gpt-5.5	gpt-5.4	多让它列 tradeoff 和未知项
普通 bug fix	gpt-5.4	gpt-5.4-mini	先要复现和最小验证
安全 review	gpt-5.5	gpt-5.3-codex	不要为了省成本牺牲召回
文档整理和小重构	gpt-5.4-mini	gpt-5.4	约束输出格式和证据
子代理探索	gpt-5.4-mini	gpt-5.4	子代理只回答证据, 不做主线决策
GUI / computer use	gpt-5.5	gpt-5.4	需要更强视觉和工具精度

思考强度建议:

- low: 快速回答、轻量编辑、已知路径的小任务。
- medium: 默认起点，平衡质量、延迟和成本。
- high: 难 bug、跨模块变更、评审、迁移。
- xhigh: 架构级重构、安全关键、复杂长期任务。

不要迷信高 reasoning。高 reasoning 在目标模糊、上下文污染、验证标准不清时，可能只是更认真地走偏。先把任务边界写清楚，再提高 reasoning。

4. 从 everything-claude-code 迁移来的核心思想

everything-claude-code 的价值不在于 Claude Code 专有配置，而在于一套 agent harness 设计哲学。迁移到 Codex 时，保留思想，替换实现。

ECC 思想	Claude Code 表达	Codex 原生表达
持久项目约束	CLAUDE.md / AGENTS.md	AGENTS.md + .codex/AGENTS.md
按需加载知识	Skills	Codex Skills
确定性护栏	Hooks / rules	Codex Hooks + .rules + sandbox/approval
结构化外部工具	MCP	Codex config.toml 中的 MCP servers
并行探索	Subagents / multi instances	Codex subagents + worktrees + cloud tasks
长任务上下文管理	Compact / session notes	Codex compaction + memories + Stop hooks
验证闭环	test/lint/eval hooks	codex exec、hooks、local environments、GitHub review
安全优先	AgentShield / sandboxing	sandbox_mode、approval_policy、rules、hooks、Codex Security
自我进化	Continuous learning	memories + skill extraction + session review

一个重要更新: everything-claude-code 的 Codex 说明里保留了 "Codex 缺少 Claude-style hook parity" 的判断，但 OpenAI 当前官方文档已经给出 Codex Hooks，并列出 SessionStart、PreToolUse、PermissionRequest、PostToolUse、UserPromptSubmit、Stop 等事件。因此，Everything CodeX 不应该继续把 hooks 视为缺口，而应该把 hooks 纳入 Codex 原生高级能力。

5. 你的 Codex 工作台

基础安装:

```
npm i -g @openai/codex
codex --version
codex login status
```

CLI 核心命令:

```
codex          # 打开交互 TUI
codex "解释这个代码库" # 带初始 prompt 启动
codex -m gpt-5.5      # 指定模型
codex -p strict        # 使用配置 profile
codex --search         # 开启 live web search
codex exec "总结仓库结构" # 非交互运行
codex exec --json "分析失败测试" | jq
codex review          # 非交互 code review
codex mcp list        # 查看 MCP
codex plugin list     # 查看插件
codex cloud           # 浏览 Codex Cloud 任务并本地应用改动
```

建议目录结构:

```
repo/
  AGENTS.md
  .codex/
    config.toml
    AGENTS.md
    hooks.json
    rules/
      default.rules
  agents/
    explorer.toml
    reviewer.toml
    docs-researcher.toml
  local-environments/
  skills/
    your-domain-skill/
      SKILL.md
    scripts/
    references/
```

最低配置示例:

```
# .codex/config.toml
model = "gpt-5.5"
approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "live"

[features]
hooks = true
memories = true

[profiles.strict]
approval_policy = "on-request"
sandbox_mode = "read-only"
web_search = "cached"

[profiles.full]
approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "live"

[agents]
max_threads = 6
max_depth = 1
```

关键原则:

- 默认 **workspace-write**, 不要默认 **danger-full-access**。
- 复杂任务开新 thread, 不要把一个项目所有事情塞进一个超长 thread。
- 每个仓库都要有 **AGENTS.md**。
- 每个任务都要有可运行的验证命令。

- 每次任务结束要把可复用经验归档。

6. AGENTS.md: 你的 Codex 宪法

AGENTS.md 是 Codex 的项目级长效上下文。它不应该写成百科，而应该写成操作规则。

推荐结构:

```
# AGENTS.md

## Repository Expectations
- Prefer small, focused changes.
- Do not rewrite unrelated files.
- Run `npm test` and `npm run lint` before proposing completion.
- Use `rg` for search.
- Do not commit or push unless explicitly asked.

## Architecture
- API routes live in `src/server/routes`.
- Domain logic belongs in `src/domain`.
- UI components must use the existing design system.

## Review Guidelines
- Treat auth, payment, data deletion, and PII handling as high risk.
- Flag missing tests for any behavior change.
- Lead with concrete file/line findings.

## Verification
- Unit tests: `npm test`
- Typecheck: `npm run typecheck`
- E2E: `npm run e2e`

## Done Definition
- Code compiles.
- Tests pass or failures are explained with evidence.
- Diffs are scoped to the requested task.
- Public behavior changes are documented.
```

写 AGENTS.md 的常见错误:

- 把大量历史背景塞进去，导致每次启动都污染上下文。
- 写愿景口号，不写具体约束。
- 没有验证命令。
- 没有 review guidelines。
- 把个人偏好和团队必须遵守的规则混在一起。

分层方法:

- `~/ .codex/AGENTS.md`: 个人全局偏好，比如回答风格、默认安全边界。
- `repo/AGENTS.md`: 仓库级工程规则。
- `repo/package/AGENTS.md`: 子目录特殊规则，比如 payments、mobile、infra。
- `.codex/AGENTS.md`: Codex 专用补充，不要替代根 AGENTS.md。

7. config.toml: 把默认行为工程化

config.toml 控制模型、权限、沙箱、profiles、MCP、features、agents、hooks 等。官方文档说明用户级配置在 `~/.codex/config.toml`，项目级覆盖在 `.codex/config.toml`，项目级配置只有在信任项目后才加载。

高水平用户通常至少维护三个 profile:

```
[profiles.explore]
approval_policy = "on-request"
sandbox_mode = "read-only"
web_search = "live"

[profiles.build]
approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "live"

[profiles.automation]
approval_policy = "never"
sandbox_mode = "workspace-write"
web_search = "cached"
```

使用方式:

```
codex -p explore "梳理支付模块调用链，不要修改文件"
codex -p build "实现退款状态机并补测试"
codex exec -p automation --json "从 CI 日志中提取失败原因"
```

配置原则:

- 把危险能力做成显式 profile，不要写成默认。
- 把项目内 MCP 放在项目 `.codex/config.toml`，把私人 token 放在用户级环境变量或 secret 管理里。
- 不要盲目启用所有 MCP。每个工具描述都会消耗上下文和注意力。
- 用 `--config key=value` 做临时实验，不要频繁改全局配置。

8. Rules: 命令权限的确定性边界

Rules 控制 Codex 哪些命令可以越过沙箱运行，哪些要提示，哪些永远禁止。它不是 prompt，不依赖模型自觉。

示例:

```
# .codex/rules/default.rules
prefix_rule(
  pattern = ["gh", "pr", "view"],
  decision = "prompt",
  justification = "Viewing PRs is allowed with approval",
  match = [
    "gh pr view 123",
    "gh pr view 123 --json title,body",
  ],
  not_match = [
    "gh pr --repo org/repo view 123",
  ],
)

prefix_rule(
  pattern = ["rm", "-r"],
  decision = "forbidden",
  justification = "Never allow recursive forced deletion through Codex; ask the user for an explicit manual cleanup plan.",
)
```

Rules 适合:

- 永远禁止的危险命令。
- 总是需要确认的外部副作用。
- CI、GitHub、deploy、数据库迁移等高风险命令前缀。

Rules 不适合:

- 长篇工作流说明。
- 风格偏好。
- 只有某些任务才需要的知识。

高水平做法:

1. 先用默认交互审批跑几周。
2. 把反复批准的安全命令提炼成 allow 或 prompt rule。
3. 把绝不应该发生的命令提炼成 forbidden rule。
4. 为每条 rule 写 match/not_match 样例。

9. Hooks: Codex 的确定性自动化

Codex Hooks 可以在 agent lifecycle 中运行脚本。官方列出的用途包括日志、阻止 prompt 中误粘 API key、自动总结对话生成记忆、turn 停止时跑验证、按目录注入上下文。

推荐从 5 类 hooks 入手:

Hook	目标	示例
SessionStart	加载项目状态	读取 docs/current-state.md 并补充上下文
UserPromptSubmit	输入防护	阻止 secret、token、私钥被粘进 prompt
PreToolUse	危险命令拦截	拦截 <code>rm -rf</code> 、生产库命令、未审批 deploy
PostToolUse	运行后记录	记录测试命令结果、统计失败模式
Stop	结束时验证	自动提示未跑测试、生成 session summary

示例 hooks.json:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "python3 .codex/hooks/block_dangerous_shell.py",
            "statusMessage": "Checking shell command"
          }
        ]
      }
    ]
  }
}
```

```

}
],
"Stop": [
{
  "matcher": "*",
  "hooks": [
    {
      "type": "command",
      "command": "python3 .codex/hooks/session_summary.py",
      "statusMessage": "Writing session summary"
    }
  ]
}
]
}
}
}
}

```

Hooks 的安全边界:

- Hook 是 guardrail, 不是完整安全沙箱。
- 多个匹配 hook 会并发运行, 不能假设顺序。
- 非托管 hook 要先 review 和 trust。
- Project-local hooks 只有在项目 `.codex/` layer 被信任后才加载。
- PreToolUse 不能拦截所有可能路径, 因此高风险动作还要配合 rules、sandbox、approval。

10. Skills: 把一次性 prompt 变成复用能力

Skill 是 Codex 的可复用 workflow 单元。它通常由 `SKILL.md`、脚本、参考资料和模板组成。官方文档强调, skills 使用 progressive disclosure: Codex 初始只看到 name、description、path, 真正需要时才加载完整说明。

什么时候应该创建 skill:

- 你连续 3 次向 Codex 解释同一种工作流。
- 某类任务需要固定步骤和验证命令。
- 某个领域知识很重, 不适合塞进 `AGENTS.md`。
- 任务需要脚本、模板、参考文档配合。
- 你希望团队成员得到一致输出。

Skill 模板:

```

---
name: refund-state-machine-review
description: Use when changing refund state machine logic, settlement flows, or payment
reversal behavior.
---

# Refund State Machine Review

## When to Use
- Any change touching refund state transitions.
- Any change touching payment reversal, settlement, or reconciliation.

## Required Context
- Read `docs/payments/refund-state-machine.md`.
- Inspect current tests under `tests/payments/refund`.
- Identify all external side effects.

```

```
## Workflow
1. Map current state transition path.
2. Write or update failing tests before changing implementation.
3. Make the smallest implementation change.
4. Run unit tests and integration tests.
5. Produce a risk summary.

## Verification
- `npm test -- tests/payments/refund`
- `npm run typecheck`

## Output
- Changed files
- Tests run
- Remaining risks
- Rollback notes
```

Skill 分层:

- Standards skill: 编码规范、错误处理、测试标准。
- Workflow skill: TDD、PR review、发布前检查、事故复盘。
- Domain skill: 支付、库存、搜索、推荐、合规。
- Integration skill: GitHub、Jira、Slack、数据库、内部系统。
- Teaching skill: 把你的实践流程包装成课程演示脚本。

不要做:

- 不要把所有知识塞进一个 mega skill。
- 不要把 "必须永远遵守" 的规则放进 skill，它应该在 AGENTS.md 或 rules。
- 不要创建没有触发边界的泛泛 skill。
- 不要只写原则，不写执行步骤和验证命令。

11. MCP、CLI、API、Skill 的选择

everything-claude-code 的 surface selection 思想非常适合 Codex。决策顺序如下:

1. 必须每次都生效，且不需要模型判断: 用 rule 或 AGENTS.md。
2. 是多步骤工作流或领域 playbook: 用 skill。
3. 需要结构化工具输入输出、跨客户端重复调用、长驻 server: 用 MCP。
4. 是一次性本地确定性动作: 用 CLI 或 repo script。
5. 是某个 workflow 中很窄的远程动作: 直接在 skill/script 里调 API。

实践例子:

场景	最佳 surface	
禁止删除生产数据	rule + hook	
支付退款变更流程	skill	
查内部文档、Figma、浏览器、数据库	MCP / connector	

场景	最佳 surface	
生成 coverage 报告	repo script + skill	
GitHub PR review	GitHub integration + AGENTS.md review guidelines	
每日 bug triage	Codex App automation + skill	
一次性读取 CI 日志	`gh run view ...`	codex exec ...`

MCP 管理原则:

- 少即是多。每个项目只启用真正需要的 MCP。
- 优先启用高价值工具: GitHub、docs lookup、browser/playwright、database read-only、Figma、Vercel。
- 能用 CLI 简单完成的, 不一定要 MCP。
- 对写操作 MCP 保持默认只读, 外部副作用必须显式确认。

12. Subagents: 从单线程聊天到并行工程

Codex subagents 适合高度可并行的任务。官方文档说明 Codex 可以并行生成专门 agent, 然后汇总结果; 但 Codex 只会在你明确要求时 spawn subagent。

适合 subagents 的任务:

- 大代码库探索: A 查路由, B 查数据模型, C 查测试。
- PR review: A 查正确性, B 查安全, C 查测试缺口。
- 文档核验: A 查官方 docs, B 查代码实现, C 查 changelog。
- 多模块迁移: A 迁移 API 层, B 迁移 UI 层, C 迁移测试。

不适合 subagents 的任务:

- 下一步完全依赖探索结果的任务。
- 写同一个文件的多个实现者。
- 范围模糊、没有交付物的任务。
- 你无法 review 子代理结果的任务。

推荐角色:

```
#.codex/agents/explorer.toml
model = "gpt-5.4-mini"
model_reasoning_effort = "medium"
sandbox_mode = "read-only"

developer_instructions = """
Stay read-only. Trace files, symbols, tests, and runtime paths.
Return evidence with paths and line references.
Do not propose broad refactors.
"""

#.codex/agents/reviewer.toml
```

```
model = "gpt-5.5"
model_reasoning_effort = "high"
sandbox_mode = "read-only"

developer_instructions = """
Review for correctness, security, behavior changes, and missing tests.
Findings first. Use file and line evidence.
Avoid style-only comments unless they hide real risk.
"""
```

父 agent 分配任务时要写清:

- 子代理只读还是可写。
- 拥有哪些文件或模块。
- 输出格式是什么。
- 是否允许建议代码修改。
- 何时停止。

13. Worktrees 和并行任务

Codex App 原生支持 worktrees。官方文档说明, worktrees 让 Codex 在同一项目中运行多个独立任务, 避免互相干扰; automations 也可以跑在 dedicated background worktree 上。

三种并行层级:

1. 同一 thread 内 subagents: 适合读、查、评审。
2. 同一 repo 多 worktrees: 适合并行实现不同模块。
3. Codex Cloud 多任务: 适合后台 issue、PR、CI、文档任务。

建议模式:

main thread:
控制任务目标、接受结果、做最终集成

worktree-a:
实现 feature A

worktree-b:
修复 test suite

subagent-explorer:
只读追踪代码路径

subagent-reviewer:
对最终 diff 做 review

并行守则:

- 并行前先拆文件所有权。
- 不要让两个 agent 同时改同一个文件。
- 每个 worktree 都有自己的验证命令。
- 父 thread 负责最终整合。

- 子任务结束要产出 changed files、tests run、risks。

14. Prompting: outcome-first, 而不是 step-by-step 控制欲

官方 GPT-5.5 指南强调 outcome-first prompts: 讲清结果、成功标准、允许的副作用、证据规则和输出形状。不要在有必要时规定每一步怎么做。

最小高质量 prompt:

目标:

修复登录后刷新页面会丢失 session 的问题。

上下文:

- 入口: src/auth/session.ts, src/routes/login.ts
- 复现: 登录成功后刷新 /dashboard, 会跳回 /login
- 预期: 刷新后仍保持登录态

约束:

- 只做最小修复
- 不改变 token 格式
- 不改公共 API
- 不提交代码

验证:

- 先写或定位能复现的测试
- 运行 npm test -- auth
- 运行 npm run typecheck

输出:

- 根因
- 修改了哪些文件
- 测试结果
- 剩余风险

复杂任务 prompt:

请先进入 read-only 探索模式:

1. 梳理订单取消流程的真实调用链。
2. 找出状态机、库存回滚、支付取消、通知发送的关键文件。
3. 不要修改文件。
4. 输出: 调用链、关键风险、建议拆分计划、需要我确认的问题。

等我确认计划后, 再进入实现。

评审 prompt:

请对当前 diff 做 owner-level review。

优先级:

1. 正确性和行为回归
2. 安全和权限
3. 并发、事务、幂等
4. 测试缺口

不要输出风格建议, 除非它会导致真实 bug。

每条 finding 必须包含文件、行号、影响、建议修复。

坏 prompt 的特征:

- "帮我优化一下"。
- "看看有没有问题"。
- "重构一下这个模块"。

- 没有路径、没有验证、没有 done definition。
- 让 Codex 同时探索、设计、实现、发布、写总结，但不给停顿点。

15. Codex CLI 深度掌握

CLI 是 Codex 的高阶入口。你要练到能把它嵌入 shell、CI、日志分析和本地开发流程。

常用模式:

交互模式

```
codex -C /path/to/repo
codex -m gpt-5.5 --search "调研这个 repo 的架构并提出测试补强计划"
```

非交互模式

```
codex exec "summarize the repo structure"
codex exec --sandbox workspace-write "fix lint errors and run tests"
codex exec --json "triage failing tests" | jq
codex exec -o result.md "write a concise architecture brief"
```

stdin 管线

```
npm test 2>&1 | codex exec "找出最可能的失败根因，给出最小修复建议"

gh run view 123456 --log \
| codex exec "用 5 条 bullet 总结 CI 失败原因和下一步" \
> ci-summary.md
```

结构化输出

```
codex exec "Extract project metadata" \
--output-schema ./schema.json \
-o ./project-metadata.json
```

安全运行

```
codex exec --sandbox read-only "找出潜在安全问题，不要修改"
codex exec --sandbox workspace-write "修复测试，不要访问网络"
codex exec --ignore-user-config "在干净配置下复现问题"
```

高级指标:

- 你能否让 `codex exec --json` 的输出进入自己的 dashboard。
- 你能否用 schema 让 Codex 产出稳定机器可读结果。
- 你能否把 CI log、test log、coverage report、Git diff 作为 stdin 输入。
- 你能否把重复命令包装成 repo script 或 skill。

16. Codex App、IDE、Cloud 怎么分工

任务	首选 surface	原因
边看 diff 边指导修改	Codex App	review pane 和 inline feedback 更自然
正在编辑一个函数	IDE Extension	open files 自动进上下文
后台跑 issue 修复	Codex Cloud	不阻塞本地工作
手机上跟进长任务	Remote mobile	批准动作、看输出、补指令
多项目并行	Codex App	project sidebar 和 threads
CI 自动总结	CLI exec	shell 管线和 JSONL
GitHub PR review	GitHub integration	@codex review 和自动 review
GUI 复现 bug	Codex App Computer Use	需要真实 UI 操作

Codex App 的三个关键能力:

- Worktrees: 并行任务不污染本地 checkout。
- Review pane: 以 Git diff 为中心做反馈、stage、revert。
- Automations: 定时后台任务、Triage inbox、结合 skills 做复杂周期任务。

IDE Extension 的关键能力:

- 适合局部理解、局部修改、打开文件上下文强相关任务。
- 适合你仍然主导编码，但让 Codex 处理解释、补测试、局部 patch。

Cloud 的关键能力:

- 适合明确的后台任务。
- 适合 issue、PR、CI、依赖升级、文档同步。
- 不适合需要你即时大量互动的探索型任务。

17. GitHub 工作流

Codex GitHub integration 支持 @codex review、自动 review、@codex fix 和其他 PR comment 触发的 cloud task。官方说明 Codex review 会聚焦严重问题，并按仓库里的 AGENTS.md review guidance 执行。

建议在 AGENTS.md 加:

```
## Review guidelines
- Flag auth bypass, PII logging, data deletion, permission regressions as P1.
- Verify every behavior change has a test or a documented reason.
- Treat transaction, idempotency, retry, and concurrency issues as high priority.
- Do not comment on formatting if formatter handles it.
```

PR 流程:

- 1. 人类或 Codex 完成实现
- 2. 本地跑测试
- 3. 创建 PR
- 4. 评论 @codex review
- 5. 对 P0/P1 finding 评论 @codex fix the P1 issue
- 6. 人类 review Codex patch
- 7. CI 绿后 merge

高水平用法:

- 用 AGENTS.md 控制 review 风格。
- 对一次性重点直接写在 PR comment: @codex review for auth regressions。
- 把 Codex review 当第二道高信号 review，不要替代 owner review。
- 对自动 review 设置清楚触发条件，避免噪音。

18. 验证闭环和 Evals

Top 1% 用户不相信 "看起来可以"。他们要求 Codex 交付证据。

每个任务至少有 4 层验证:

- 1. 静态验证: lint、typecheck、format。
- 2. 单元验证: 目标模块测试。
- 3. 行为验证: 集成测试、E2E、手动复现。
- 4. 回归验证: 与旧行为对比、diff review、风险清单。

Prompt 里的验证写法:

- 验证要求:
- 如果已有测试能覆盖，先说明测试名。
 - 如果没有测试，先补一个失败测试。
 - 修改后运行最小测试集。
 - 如果测试无法运行，说明阻塞原因和你已经验证过的替代证据。

Eval 设计:

Eval 类型	用途
Checklist eval	适合 PR review、发布前检查
Golden task eval	适合固定任务，比如 "修复某类 lint"
Regression eval	适合长期追踪 prompt/skill 变更是否退化
Pass/fail script eval	适合可自动判定的测试
Human rubric eval	适合架构文档、方案质量、讲课材料

衡量 Codex 使用水平的指标:

- 一次任务平均交互轮数。
- 首次 patch 通过率。

- 任务结束时可验证证据比例。
- 需要人工手工修补的比例。
- 复用 skill 后任务成本下降比例。
- Codex 引入回归的比例。
- 你每周沉淀的新 skill/rule/hook 数。

19. Context 与 Memory 管理

everything-claude-code 强调 context rot、session summary、strategic compact、continuous learning。Codex 当前也有 memories、compaction、hooks 和 AGENTS.md 分层。

上下文分四类:

类型	放哪里
必须遵守的规则	AGENTS.md / rules
可复用 workflow	skill
自动化护栏	hook
稳定偏好和历史经验	memories
临时探索结果	thread summary / task note
机器可读状态	JSON artifact / status file

不要把所有东西都塞进 AGENTS.md。AGENTS.md 是宪法，不是日志。

推荐 Stop hook 产物:

```
# Session Summary
## Goal
## Completed
## Files Changed
## Commands Run
## What Worked
## What Failed
## Open Risks
## Reusable Learning
## Candidate Skill / Rule / Hook
```

每周复盘:

1. 统计本周 Codex 完成的任务。
2. 找出重复解释超过 3 次的内容。

3. 找出反复失败的测试/命令/流程。
4. 新建或更新 1-3 个 skills。
5. 更新 AGENTS.md 中真正稳定的规则。
6. 把危险命令沉淀为 rules 或 hooks。

20. 安全与治理

Codex 能读、改、运行代码，也能通过 MCP、Computer Use、GitHub 和 cloud task 影响外部系统。越强的 agent，越需要工程化安全边界。

基本防线:

- `sandbox_mode = "workspace-write"` 作为默认。
- `approval_policy = "on-request"` 作为交互默认。
- 高风险自动化才用 `never`，且必须在隔离环境。
- 写操作 MCP 默认关掉或只读。
- 生产资源操作需要明确人类确认。
- Secrets 不进 prompt、不进日志、不进 AGENTS.md、不进 memory。

安全 checklist:

Before allowing Codex to commit/push/deploy:

- git diff reviewed
- no secrets
- no credential files changed
- tests run
- generated files intentional
- external side effects listed
- rollback path known

Computer Use 安全:

- 只给明确任务。
- 不让 Codex 在未知 app 中自由探索。
- 操作前看权限提示。
- 对浏览器、设置、支付、生产后台、账号管理类任务保持手动确认。

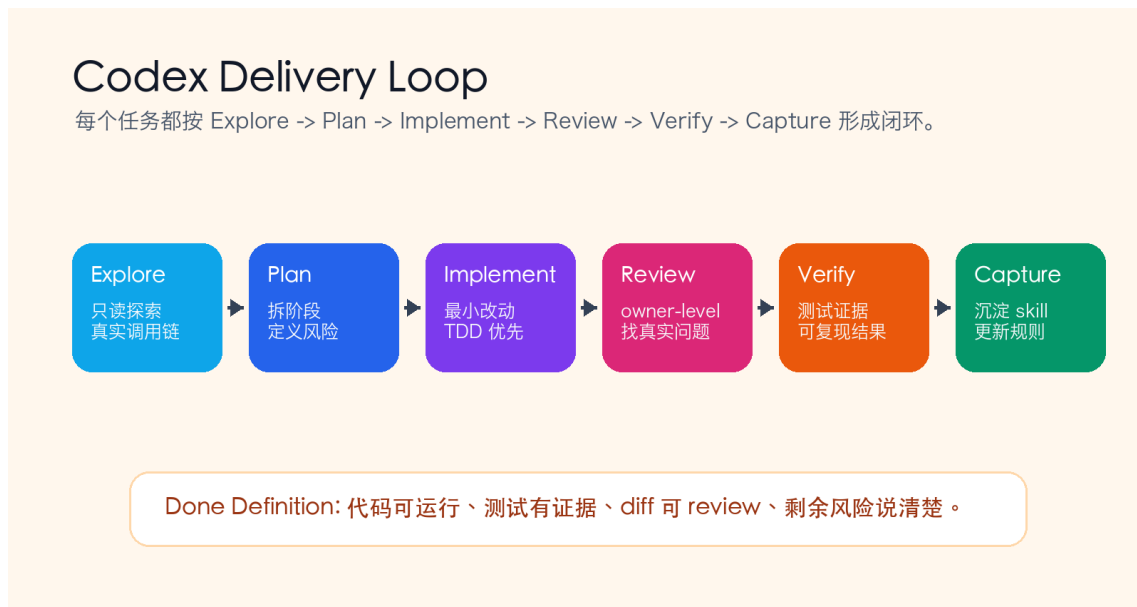
MCP 安全:

- 优先只读。
- 明确写操作权限。
- 不启用不需要的 MCP。
- 对供应链不清楚的 MCP 先隔离测试。

Hooks 安全:

- 不信任陌生 hooks。
- Hook 脚本也要 code review。
- Project-local hooks 需要信任项目后才加载，这是好事，不要绕过。

21. 经典 workflow 模板



新功能

请按 Explore -> Plan -> Implement -> Review -> Verify 执行。
 先只读探索，不改文件。
 产出计划后等我确认。
 确认后按 TDD 修改。
 完成后运行测试，并输出 changed files、test results、risks。

Bug 修复

先复现 bug。
 如果没有测试，请补一个最小失败测试。
 只做最小修复。
 不要顺手重构。
 修复后运行目标测试和 typecheck。
 输出根因、修复点、为什么不会引入回归。

大重构

先生成重构地图：

- 当前调用链
- 风险点
- 可拆分阶段
- 每阶段验证命令
- 回滚点

不要直接改代码。
 计划确认后，每次只做一个阶段。

PR Review

Review current diff like a repository owner.
Prioritize correctness, security, behavior regressions, and missing tests.
Findings first.
No style-only comments.
Every finding needs file/line, impact, and suggested fix.

文档/调研

只使用 primary sources 或仓库内真实文件。
所有关键判断给出出处。
把不确定项列为 open questions。
输出可直接放进 docs/ 的 Markdown。

前端实现

实现真实可用界面，不做 landing page 除非明确要求。
遵循现有 design system。
用 Playwright 或 browser 截图验证桌面和移动端。
检查文字不溢出、元素不重叠、交互状态完整。

22. 自动化: 从手工 prompt 到工程流水线

Codex exec 适合自动化:

```
gh run view "$RUN_ID" --log \
| codex exec --json "分类 CI 失败原因，输出最小修复建议" \
> artifacts/ci-triage.jsonl
```

Codex App automation 适合周期任务:

- 每天扫描 bug backlog，输出优先级。
- 每天检查 flaky tests。
- 每周检查依赖升级风险。
- 每周总结代码库热点文件。
- 每天跑 docs drift 检查。

Hook 适合确定性动作:

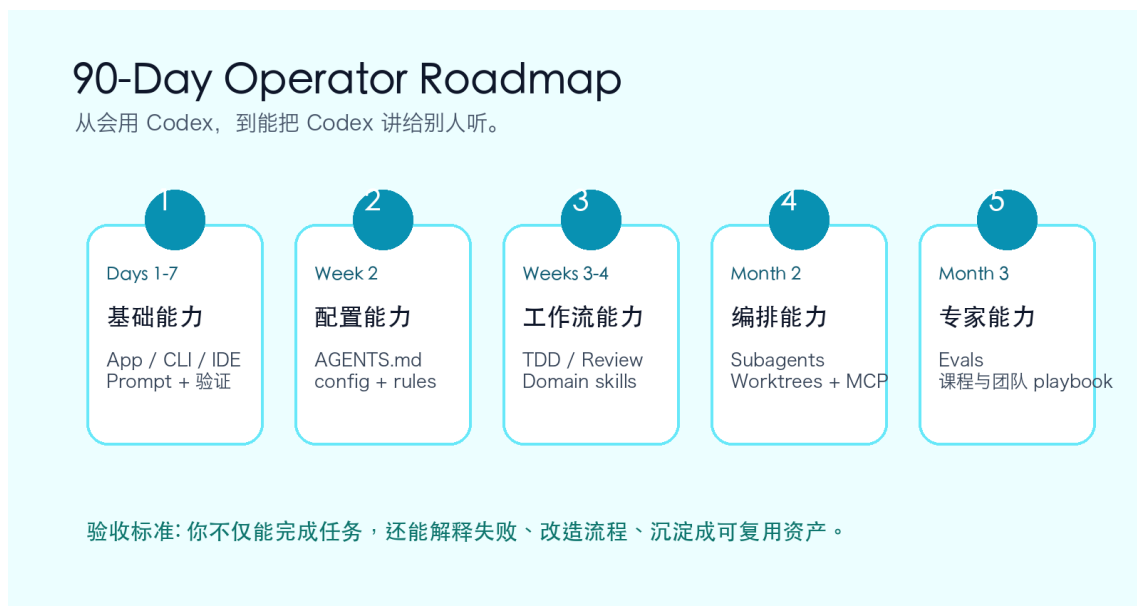
- Stop 时提醒未跑测试。
- PreToolUse 阻止危险命令。
- UserPromptSubmit 检查 secrets。
- SessionStart 加载当前迭代状态。

Skill 适合复杂 playbook:

- CI 失败 triage。
- 安全 review。

- 发布检查。
- 客户问题定位。
- 数据分析报告生成。

23. 90 天训练计划



第 1-7 天: 基础能力

目标: 熟练使用 Codex App、CLI、IDE extension。

每日任务:

- 用 Codex 理解一个陌生模块。
- 用 Codex 修一个小 bug。
- 用 Codex 补一个测试。
- 每次都要求 changed files、tests run、risks。

验收:

- 你能解释 Codex 各 surface 的适用场景。
- 你能写出一个合格的任务 prompt。
- 你的每个任务都有验证证据。

第 2 周: 配置能力

目标: 建立个人和项目级默认行为。

任务:

- 写 `~/ .codex/AGENTS.md`。
- 给一个真实 repo 写 `AGENTS.md`。
- 配置 `.codex/config.toml` profiles。
- 配置至少 2 条 rules。

验收:

- Codex 启动后能自动遵守仓库规则。
- 危险命令不会直接执行。

第 3-4 周: workflow 能力

目标: 把常用任务沉淀成 skills。

任务:

- 写 TDD skill。
- 写 PR review skill。
- 写安全 review skill。
- 写一个你业务领域的 domain skill。

验收:

- 同类任务耗时下降。
- 输出格式稳定。
- 新人能用你的 skill 产出相似质量。

第 2 个月: 编排能力

目标: 掌握 subagents、worktrees、MCP、hooks。

任务:

- 为 repo 配 explorer/reviewer/docs-researcher 三个 agent。
- 用 worktree 并行两个独立任务。
- 接入一个 MCP。
- 写一个 Stop hook。
- 写一个 PreToolUse hook。

验收:

- 你能并行推进任务但不冲突。
- 你能解释每个 surface 的成本和风险。

第 3 个月: 专家能力

目标: 形成可教学体系。

任务:

- 建立 10 个代表性 Codex eval tasks。
- 形成 1 小时课程。
- 做 3 个 live demo。
- 建立团队 Codex playbook。
- 给一个真实 PR 做 Codex review 对比实验。

验收:

- 你能现场解释 Codex 失败原因。
- 你能把失败转化为 skill/rule/hook/config 改进。
- 你能指导别人从 prompt 用户升级为 Codex operator。

24. Top 1% 能力评分表

能力	1 分	3 分	5 分
Prompt	模糊提问	有目标和上下文	有目标、约束、验证、输出格式
Context	让 Codex 自己猜	提供路径	分层提供 AGENTS、skill、文件、证据
Verification	看结果	跑测试	自动化验证 + 风险说明
Config	默认配置	有 profile	配置、rules、hooks、MCP 可解释
Skills	不会写	有 1-2 个	高频任务 skill 化并能评估
Subagents	不使用	偶尔探索	按文件所有权并行协作
Security	靠手感	有审批	沙箱、rules、hooks、secret 防护
Automation	手工操作	用 exec	CI、App automation、JSONL、schema
Teaching	只会自己用	能演示	能诊断他人 workflow 并改造系统

Top 1% 门槛:

- 总分 36/45 以上。
- 你能完成一个从 0 到 PR 的端到端演示。
- 你有自己的 AGENTS.md 模板、config 模板、3 个以上 skills、2 个以上 hooks、1 套 review rubric。
- 你能解释 Codex 与 Claude Code、Cursor、Copilot 的 workflow 差异。

25. 对外授课大纲

课程 1: Codex 是什么

- Codex 能力地图。
- App / CLI / IDE / Cloud / GitHub / Mobile 的分工。
- 模型选择和 reasoning effort。
- 现场演示: 让 Codex 理解一个陌生 repo。

课程 2: 写出稳定任务

- Outcome-first prompt。
- 上下文、约束、验证、输出格式。
- 常见坏 prompt 改写。
- 现场演示: 修一个 bug 并补测试。

课程 3: 工程化 Codex

- AGENTS.md。
- config.toml。
- rules。
- hooks。
- skills。
- 现场演示: 从一次 prompt 沉淀成 skill。

课程 4: 并行与自动化

- Subagents。
- Worktrees。
- Codex exec。
- App automations。
- GitHub review。
- 现场演示: 并行探索 + 实现 + review。

课程 5: 安全、评估和团队落地

- 沙箱与审批。
- MCP 风险。
- Secret 防护。
- Evals。

- 团队 playbook。
- 现场演示: 用 Codex review 找 P1 问题。

26. 你的个人 Codex Playbook

建议在自己的知识库维护:

```
Codex/  
00-Start-Here.md  
01-AGENTS-template.md  
02-config-template.toml  
03-prompt-patterns.md  
04-skills/  
05-hooks/  
06-rules/  
07-evals/  
08-course-materials/  
09-session-summaries/
```

每次高价值任务结束后, 问自己:

- 这次有没有重复解释?
- 是否应该写进 AGENTS.md?
- 是否应该变成 skill?
- 是否应该用 hook 自动化?
- 是否应该用 rule 禁止或审批?
- 是否应该加入 eval?
- 是否值得变成课程案例?

27. 最终原则

1. Codex 不是替你偷懒的工具, 而是放大你工程判断的系统。
2. Prompt 是入口, 不是核心资产。核心资产是 workflow、context、verification、memory。
3. AGENTS.md 管长期规则, skills 管按需流程, hooks/rules 管确定性边界, MCP 管结构化外部能力, subagents 管并行。
4. 任何没有验证证据的 Codex 任务都只是草稿。
5. 自动化越强, 安全边界越要工程化。
6. 你要追求的不是让 Codex 一次性写更多代码, 而是让 Codex 更稳定地交付可 review、可验证、可复用的结果。
7. 当你能把一次成功经验沉淀成下一次自动生效的系统, 你就开始进入 Top 1%。

参考来源

主要参考:

- affaan-m/everything-claude-code GitHub repo, cloned at commit **aaabe59** on 2026-05-17.
<https://github.com/affaan-m/everything-claude-code>
- OpenAI Codex overview. <https://developers.openai.com/codex>
- OpenAI Codex Quickstart. <https://developers.openai.com/codex/quickstart>
- OpenAI Codex Models. <https://developers.openai.com/codex/models>
- OpenAI GPT-5.5 guide. <https://developers.openai.com/api/docs/guides/latest-model>
- OpenAI GPT-5.5 model page. <https://developers.openai.com/api/docs/models/gpt-5.5>
- OpenAI GPT-5.3-Codex model page. <https://developers.openai.com/api/docs/models/gpt-5.3-codex>
- OpenAI Codex CLI. <https://developers.openai.com/codex/cli>
- OpenAI Codex CLI features. <https://developers.openai.com/codex/cli/features>
- OpenAI Codex non-interactive mode. <https://developers.openai.com/codex/noninteractive>
- OpenAI Codex App features. <https://developers.openai.com/codex/app/features>
- OpenAI Codex App review. <https://developers.openai.com/codex/app/review>
- OpenAI Codex App automations. <https://developers.openai.com/codex/app/automations>
- OpenAI Codex App worktrees. <https://developers.openai.com/codex/app/worktrees>
- OpenAI Codex App local environments. <https://developers.openai.com/codex/app/local-environments>
- OpenAI Codex App computer use. <https://developers.openai.com/codex/app/computer-use>
- OpenAI Codex IDE extension. <https://developers.openai.com/codex/ide>
- OpenAI Codex GitHub integration. <https://developers.openai.com/codex/integrations/github>
- OpenAI Codex remote connections. <https://developers.openai.com/codex/remote-connections>
- OpenAI Codex config basics. <https://developers.openai.com/codex/config-basic>
- OpenAI Codex config reference. <https://developers.openai.com/codex/config-reference>
- OpenAI Codex AGENTS.md guide. <https://developers.openai.com/codex/guides/agents-md>
- OpenAI Codex Rules. <https://developers.openai.com/codex/rules>
- OpenAI Codex Hooks. <https://developers.openai.com/codex/hooks>
- OpenAI Codex MCP. <https://developers.openai.com/codex/mcp>
- OpenAI Codex Skills. <https://developers.openai.com/codex/skills>
- OpenAI Codex Subagents. <https://developers.openai.com/codex/subagents>
- OpenAI Codex Memories. <https://developers.openai.com/codex/memories>
- OpenAI Codex Security. <https://developers.openai.com/codex/security>
- OpenAI Help: Using Codex with your ChatGPT plan. <https://help.openai.com/en/articles/11369540-using-codex-with-your-chatgpt-plan>